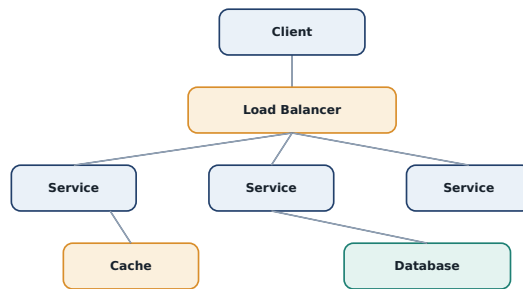


# System Design Interview Notes

Fresher se Advanced tak — scaling, databases, caching aur distributed systems, samjhaya gaya Hinglish mein, diagrams ke saath.



● Fresher ● Intermediate ● Advanced

45 Questions · 16 Diagrams · 3 Levels

# What's inside

## 01 Fresher Level

Scaling, load balancers, caching, DNS, APIs · 12 Qs

## 02 Intermediate Level

Sharding, replication, CAP, caching strategies · 17 Qs

## 03 Advanced Level

Consensus, transactions, resilience, walkthroughs · 16 Qs

## 04 Quick Revision Cheat Sheet

Trade-offs and one-liners for last-minute revision

### How to use these notes

Har question ek chhota, self-contained card hai — question bold mein, uske neeche sabse simple Hinglish explanation, aur jaha zaroorat hai wahan ek diagram. System design interviews me koi single "correct" answer nahi hota — interviewer trade-offs sunna chahta hai, isliye har topic ke "why" aur "trade-off" pe dhyaan do, sirf definition yaad mat karo.

## Core Concepts & Building Blocks

The vocabulary every fresher needs — scaling, load balancers, caching, DNS, APIs, and the shape of a basic web architecture.

### F1 What is System Design? Why does it matter in interviews?

system design basics

System Design ek process hai jisme hum decide karte hain ki ek large-scale software system (jaise Instagram, Uber, WhatsApp) ko kaise banaya jaye taaki wo **scalable**, **reliable** aur **maintainable** ho. Interviews me ye isliye important hai kyunki ye check karta hai ki tum sirf code likh sakte ho ya real-world trade-offs (cost, speed, consistency, complexity) ke beech sahi decisions bhi le sakte ho — koi “perfect” answer nahi hota, interviewer tumhara thought process dekhta hai.

### F2 Vertical scaling vs Horizontal scaling

scaling

**Vertical Scaling** (“scale up”) ka matlab hai ek hi machine ki power badhana — zyada RAM, zyada CPU laga do. Simple hai but ek limit ke baad hardware ki max capacity aa jaati hai, aur single point of failure bana rehta hai. **Horizontal Scaling** (“scale out”) ka matlab hai zyada machines add karna aur load unke beech distribute karna — theoretically unlimited scale milta hai aur fault tolerance bhi better hoti hai, but coordination aur data consistency jaisi complexity badh jaati hai.

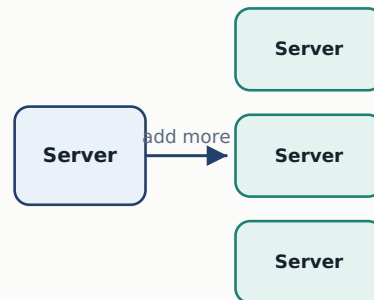
DIAGRAM

#### VERTICAL SCALING (Scale Up)



One machine gets more powerful  
— limited by hardware's max size

#### HORIZONTAL SCALING (Scale Out)



More machines share the load  
— near-unlimited, better fault tolerance

### F3 Latency vs Throughput

latency throughput

**Latency** ka matlab hai ek single request ko complete hone me kitna time lagta hai (jaise “response aane me 200ms lage”). **Throughput** ka matlab hai system kitne requests ek given time me handle kar sakta hai (jaise “1000 requests/sec”). Dono zaroori hain but trade-off ho sakta hai — batching se throughput badh sakta hai but individual request ki latency thodi badh jaati hai.

### F4 What is a Load Balancer? Why is it needed?

load balancer

Load Balancer ek component hai jo incoming traffic ko multiple servers ke beech distribute karta hai, taaki koi ek server overload na ho jaye. Fayde: **better performance** (requests parallel handle hote hain), **high availability** (ek server down ho toh LB doosre pe bhej deta hai), aur **easy scaling** (naye servers add karna aasan).

### F5 What is Caching? Why do we use it?

caching

Caching ka matlab hai frequently-accessed data ko ek fast-access storage (jaise RAM) me temporarily store karna, taaki baar-baar slow source (database/disk) tak jaana na pade. Isse response time drastically improve hota hai aur database pe load bhi kam hota hai — jaise ek popular product page ka data cache me rakh do, taaki har request pe database query na chalani pade.

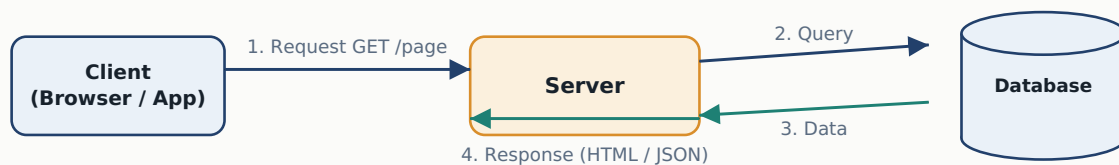
## F6 Client-Server architecture — how does a basic web request flow?

client-server request flow

Client (browser/app) ek request bhejta hai server ko (jaise “mujhe ye webpage do”), server us request ko process karta hai (database se data nikaal ke, business logic apply karke), aur response wapas client ko bhej deta hai. Ye simple request-response cycle hi modern web ka foundation hai.

DIAGRAM

### CLIENT-SERVER REQUEST/RESPONSE FLOW

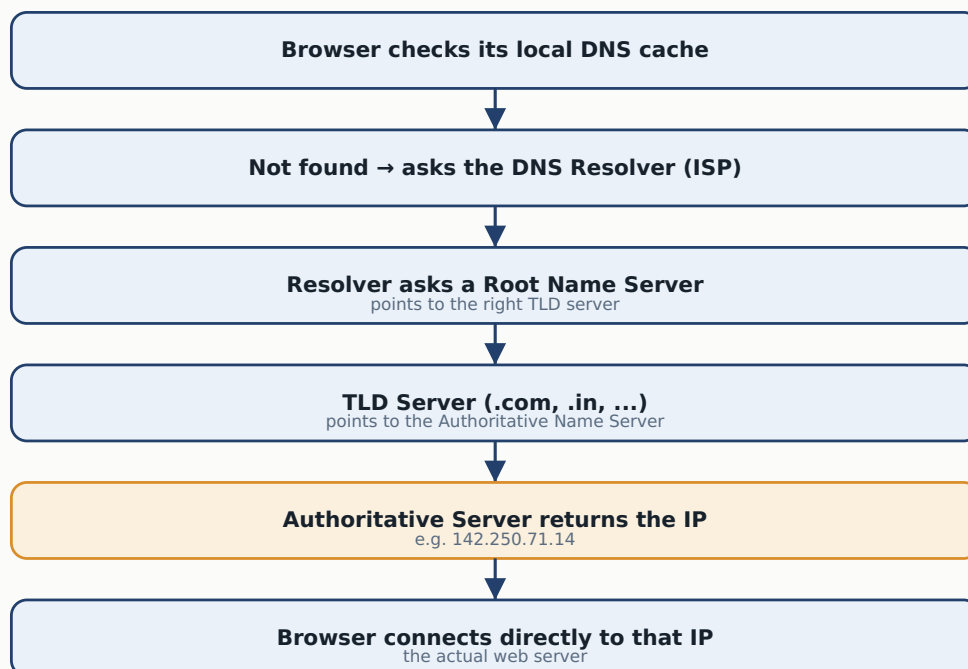


## F7 What is DNS and how does it resolve a domain to an IP?

DNS

DNS (Domain Name System) internet ka “phone book” hai — ye human-friendly domain names (jaise google.com) ko machine-readable IP addresses me convert karta hai. Browser pehle local cache check karta hai, phir DNS resolver, phir root → TLD → authoritative name servers se poochhte hue asli IP dhoondh leta hai, tab jaake browser us server se connect kar pata hai.

DIAGRAM



## F8 What is an API? REST API basics

API REST

API (Application Programming Interface) ek contract hai jisse do software components ek doosre se baat kar sakte hain, bina ye jaane ki doosra internally kaise kaam karta hai. **REST API** ek popular style hai jo HTTP methods ( GET , POST , PUT , DELETE ) use karta hai resources (jaise /users/123 ) pe operations karne ke liye, usually JSON format me data exchange karte hue.

## F9 SQL vs NoSQL databases

SQL NoSQL

**SQL (relational)** databases (MySQL, PostgreSQL) structured data ko fixed-schema tables me store karte hain aur strong consistency + complex joins support karte hain — jaha data relationships important hain wahan best. **NoSQL** databases (MongoDB, Cassandra, Redis) flexible schema rakhte hain aur horizontal scaling ke liye better designed hain — high-velocity ya massive-scale data ke liye better fit, though consistency guarantees kam strong ho sakti hain.

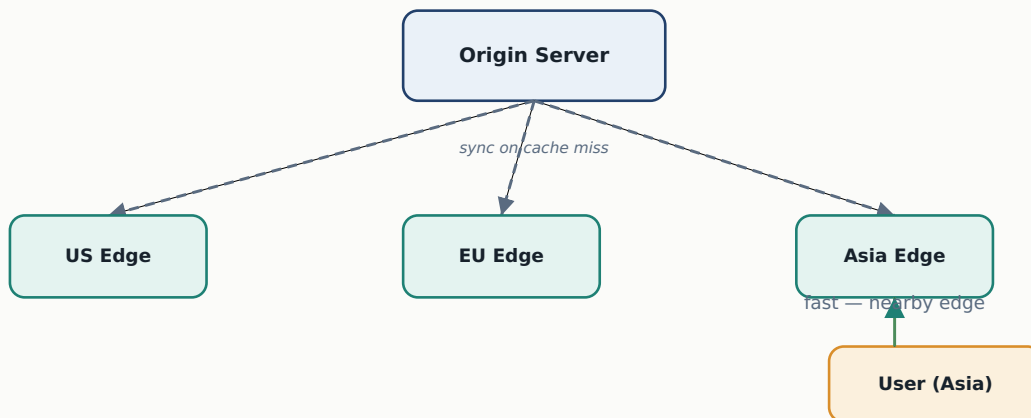
## F10 What is a CDN (Content Delivery Network)?

CDN

CDN geographically distributed servers ka network hai jo static content (images, videos, CSS/JS) ko user ke sabse paas wale location se serve karta hai, instead of har request origin server tak bhejne ke. Isse latency kam hoti hai (data kam distance travel karta hai) aur origin server pe load bhi kam ho jata hai.

DIAGRAM

### CDN — SERVE STATIC CONTENT FROM THE NEAREST EDGE



## F11 High Availability vs Fault Tolerance vs Scalability

HA fault tolerance scalability

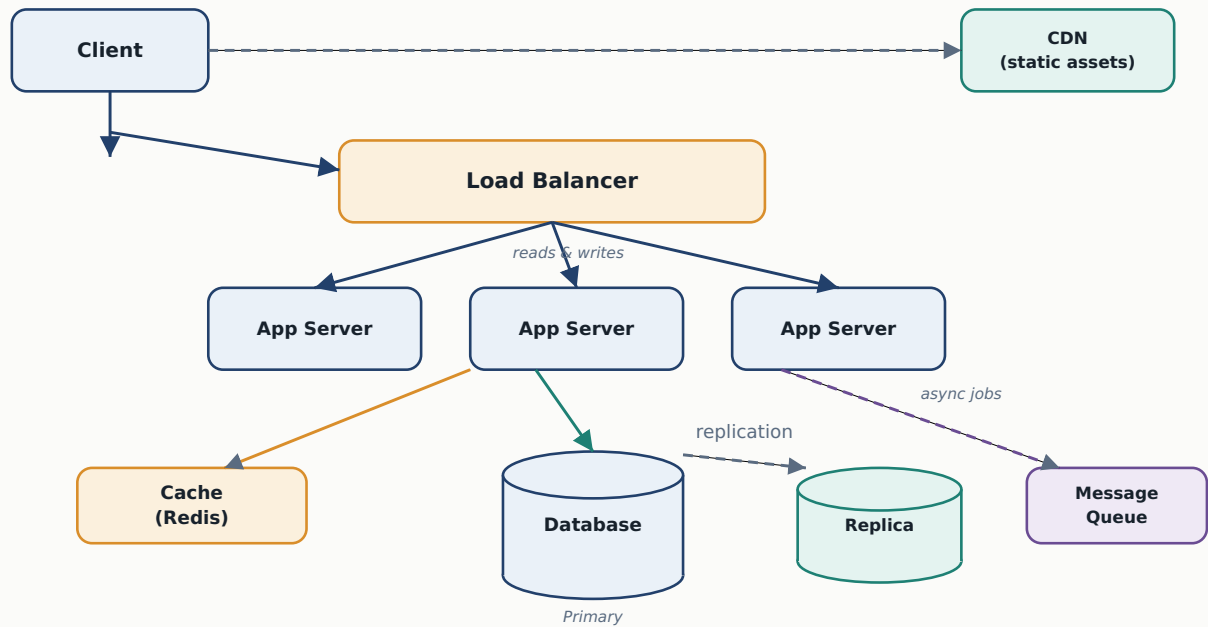
**High Availability** ka matlab hai system almost hamesha available rahe (minimal downtime) — usually redundancy se achieve hota hai. **Fault Tolerance** ka matlab hai koi component fail ho bhi jaye toh system bina kisi interruption ke chalta rahe (HA se bhi strong guarantee). **Scalability** ka matlab hai badhte load ko handle kar sake bina performance kharab kiye — teeno alag concerns hain jo saath me achhi system design banate hain.

## F12 Basic components of a scalable web architecture

architecture overview

Ek typical scalable architecture me hote hain: **Client** (browser/app), **Load Balancer** (traffic distribute karta hai), multiple **Application Servers** (business logic), a **Cache** layer (jaise Redis, fast reads ke liye), a **Database** (persistent storage, often with replicas), aur optionally a **CDN** (static content) aur **Message Queue** (async tasks). Ye sab components milke system ko fast, reliable aur scalable banate hain.

DIAGRAM



## Scaling Data & Services

Where most system design rounds live — sharding, replication, CAP theorem, caching strategies, consistent hashing, rate limiting, and microservices.

### I1 Load balancing algorithms

round robin least connections IP hash

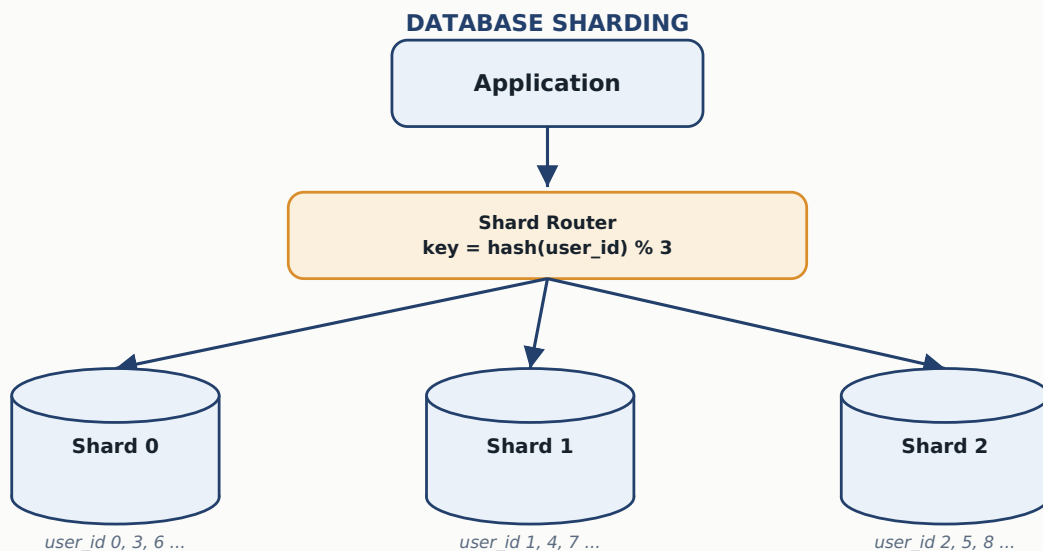
**Round Robin:** requests ko servers ke beech cyclic order me baant do (sabko baari-baari). **Least Connections:** jis server pe sabse kam active connections hain use bhejo — uneven load ke liye better. **IP Hash:** client ki IP hash karke consistently usi server pe bhejo — sticky sessions ke liye useful. **Weighted Round Robin:** servers ki capacity alag ho toh unhe weight do (powerful server ko zyada requests).

### I2 What is Database Sharding?

sharding horizontal scaling

Sharding ek technique hai jisme ek badi database ko chhote-chhote independent pieces (“shards”) me split kiya jata hai, aur har shard alag server pe store hota hai. Ye horizontal scaling ke liye zaroori hai jab ek single database server data ya load handle nahi kar pa raha. Challenge: sahi **shard key** choose karna (jaise user\_id) taaki data evenly distribute ho aur cross-shard queries minimize ho.

DIAGRAM

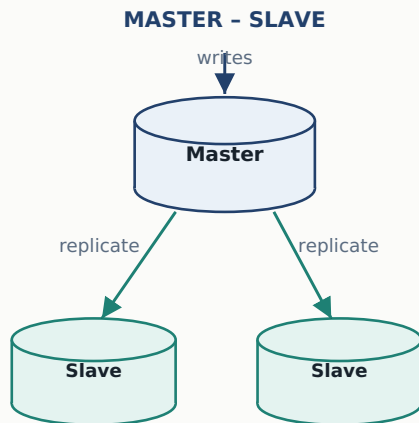


### I3 Database Replication — master-slave vs master-master

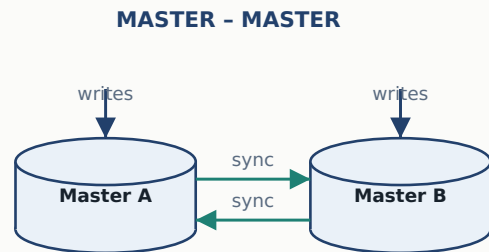
replication

**Master-Slave:** ek “master” node saari writes handle karta hai, data multiple “slave/replica” nodes pe copy hota hai jo reads serve karte hain — read-heavy systems ke liye achha, but master fail hone par failover chahiye. **Master-Master:** multiple nodes writes accept kar sakte hain aur ek doosre ko sync karte hain — better write availability but conflict resolution complex ho jata hai.

DIAGRAM



reads served by slaves; writes only via master



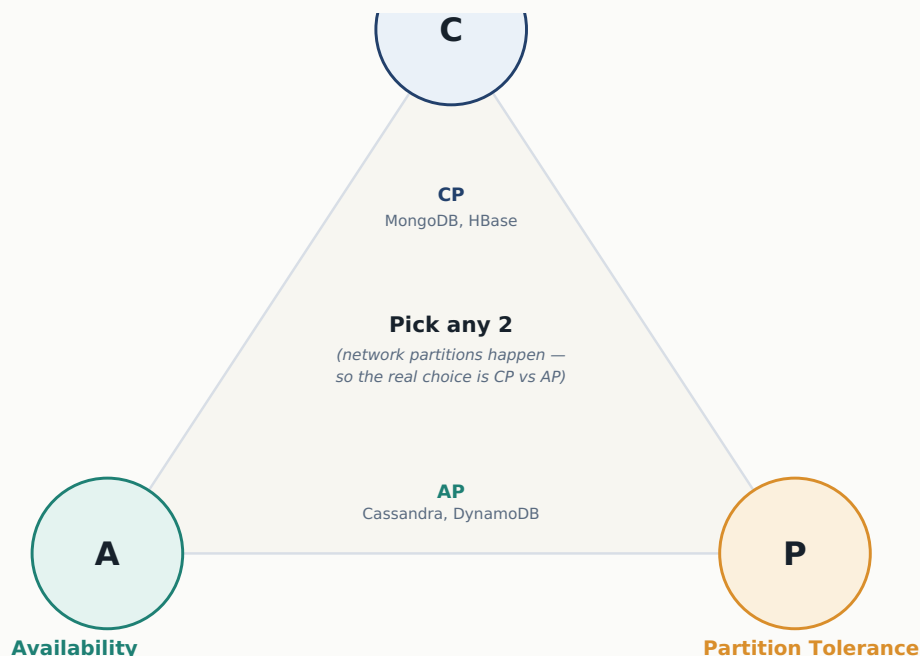
both accept writes; conflicts must be resolved between the two

### I4 What is the CAP theorem?

CAP theorem

CAP theorem kehta hai ki ek distributed system ek saath sirf 2 cheezein guarantee kar sakta hai teen me se: **Consistency** (sab nodes hamesha same data dikhayein), **Availability** (har request ko response mile), aur **Partition Tolerance** (network split hone par bhi system chalta rahe). Chunki real distributed systems me network partition kabhi na kabhi hoti hai, practically choice **CP** vs **AP** ke beech hoti hai.

DIAGRAM



## I5 ACID vs BASE

ACID BASE

**ACID** (Atomicity, Consistency, Isolation, Durability) traditional relational databases ka guarantee hai — transactions poori tarah complete ho ya bilkul na ho. **BASE** (Basically Available, Soft state, Eventual consistency) NoSQL/distributed systems ka approach hai — thoda consistency compromise karke high availability aur scalability paate hain, data eventually consistent ho jata hai.

## I6 Caching strategies: cache-aside, write-through, write-back, write-around

caching strategies

**Cache-Aside:** application pehle cache check karta hai, na mile toh database se laake cache me daal deta hai — most common pattern.

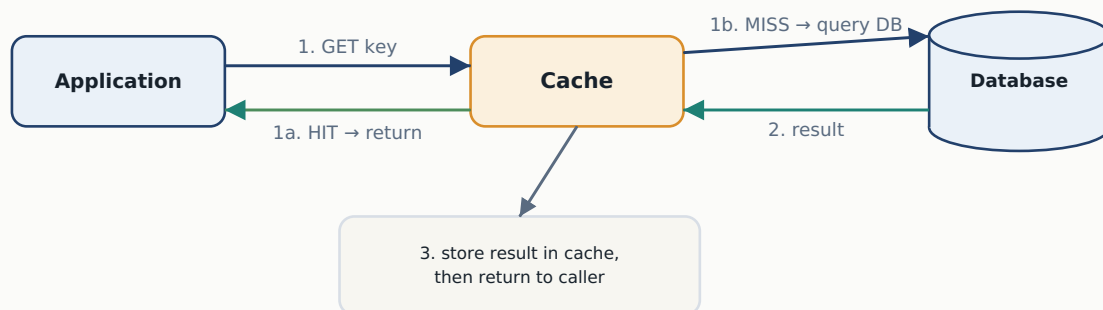
**Write-Through:** data pehle cache me likha jata hai, phir turant database me bhi — consistency achhi but writes thodi slow. **Write-**

**Back:** data pehle cache me likha jata hai, database me baad me async likha jata hai — fast but crash hone par data-loss risk. **Write-**

**Around:** data seedha database me likha jata hai, cache sirf reads ke time populate hota hai.

DIAGRAM

### CACHE-ASIDE PATTERN



## I7 Cache eviction policies: LRU, LFU, FIFO

cache eviction

**LRU** (Least Recently Used): jo sabse zyada der se use nahi hua use hatao — sabse common. **LFU** (Least Frequently Used): jo sabse kam baar access hua use hatao. **FIFO** (First In First Out): jo sabse pehle add hua tha use hatao, chahe wo abhi bhi use ho raha ho.

## I8 Reverse proxy vs forward proxy

proxy

**Forward Proxy** client ki taraf se kaam karta hai — client ke requests ko internet tak bhejta hai (client ki identity server se chhupata hai), jaise corporate VPN/proxy. **Reverse Proxy** server ki taraf se kaam karta hai — client ke requests ko backend servers tak route karta hai (backend ki identity client se chhupata hai), aur load balancing, SSL termination, caching bhi karta hai (jaise Nginx).

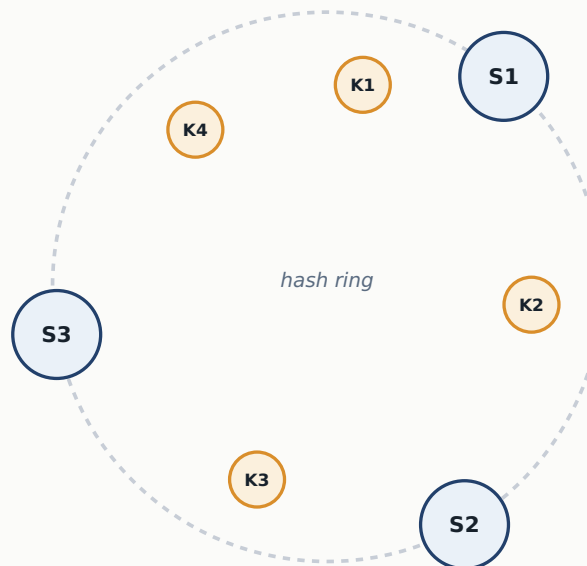
## I9 What is Consistent Hashing?

consistent hashing

Consistent Hashing ek technique hai jisse data ko multiple servers ke beech distribute karte waqt, jab koi server add/remove ho, sirf uska related data hi re-distribute karna padta hai (poora data nahi) — ek “hash ring” pe servers aur data dono ko map karke achieve hota hai. Ye distributed caching (Memcached) aur databases me scaling ke time minimal data movement ensure karta hai.

DIAGRAM

### CONSISTENT HASHING RING



*Each key is owned by the next server clockwise on the ring.  
Adding/removing one server only reshuffles its neighbouring keys.*

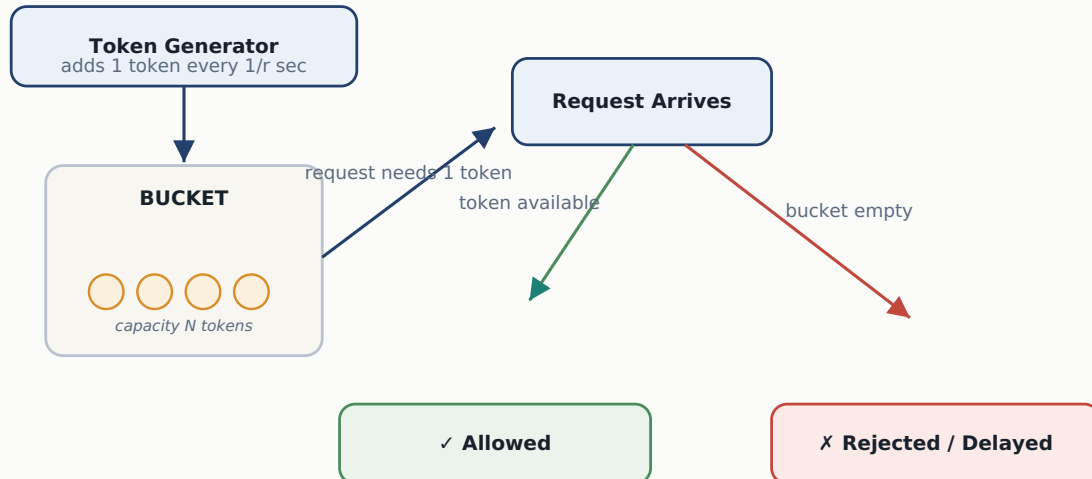
## I10 Rate limiting: token bucket vs leaky bucket

rate limiting

Rate Limiting kisi client ko ek time window me sirf fixed number of requests karne deta hai, taaki system abuse/overload na ho. **Token Bucket:** ek bucket tokens se fill hota rehta hai fixed rate pe; har request ek token use karti hai — bucket khali hone par request reject/delay hoti hai (burst traffic allow karta hai). **Leaky Bucket:** requests queue me aati hain aur fixed rate pe process hoti hain — output rate hamesha smooth/constant rehta hai, chahe input bursty ho.

DIAGRAM

### TOKEN BUCKET RATE LIMITING



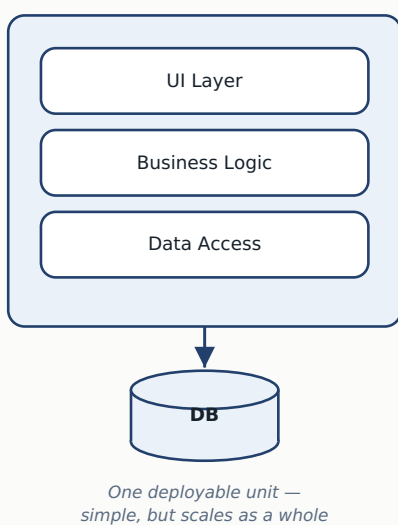
## I11 Monolith vs Microservices

monolith   microservices

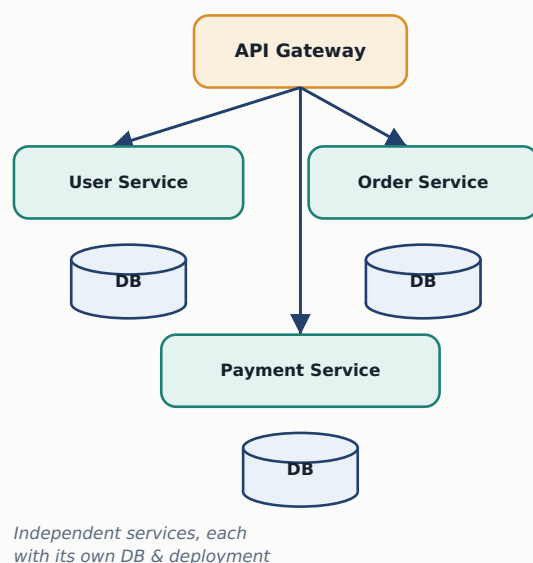
**Monolithic** architecture me poora application ek hi codebase/deployable unit hota hai — shuru me simple (easy develop/deploy/test) but scale karte waqt poore app ko scale karna padta hai. **Microservices** me application chhote, independent services me todha jata hai (har ek apna database, apna deployment) — independently scale/deploy ho sakte hain, but distributed system ki complexity (network calls, data consistency, monitoring) badh jaati hai.

DIAGRAM

### MONOLITH



### MICROSERVICES



### I12 What is an API Gateway?

API gateway

API Gateway ek single entry point hai jo client requests ko appropriate backend microservice tak route karta hai. Ye common cross-cutting concerns handle karta hai: authentication, rate limiting, request routing, response aggregation — isse individual microservices ko ye sab khud implement nahi karna padta, aur client ko sirf ek endpoint se baat karni padti hai.

### I13 What is Database indexing?

indexing

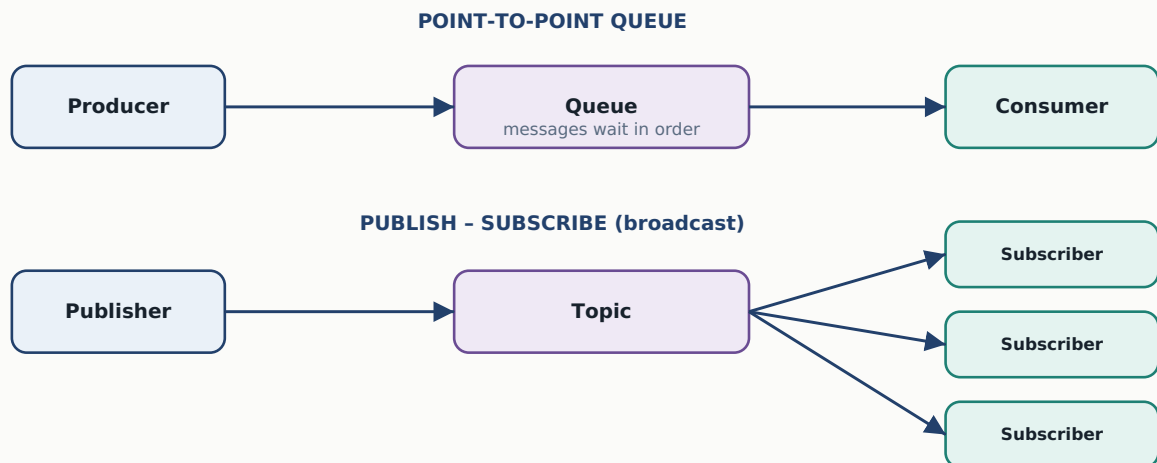
Index ek data structure hai (usually B-Tree ya Hash-based) jo database ko fast lookup karne me help karta hai, bina poori table scan kiye — jaise book ke index se directly page number pe jaa sakte ho. Trade-off: indexes read queries fast karte hain but writes (insert/update/delete) thodi slow ho jaati hain kyunki index bhi update karna padta hai, aur extra storage bhi lagti hai.

### I14 Message queues (Kafka/RabbitMQ) — why use them?

message queue Kafka RabbitMQ

Message Queue (Kafka, RabbitMQ, SQS) producers aur consumers ko decouple karta hai — producer ek message queue me daal deta hai aur turant aage badh jata hai, consumer apni speed se message process karta hai. Isse system resilient banta hai (consumer down ho toh messages queue me safe rehte hain), asynchronous processing possible hoti hai, aur traffic spikes handle karna aasan ho jata hai (buffering).

DIAGRAM



### I15 Idempotency in APIs

idempotency

Ek API operation idempotent hai agar use ek baar call karo ya multiple baar, result same rahe. Jaise “set balance = 100” idempotent hai, but “add 10 to balance” nahi hai (baar-baar call karne se badhta jayega). Idempotency zaroori hai especially retries ke liye — agar network fail ho jaye aur client dobara request bheje, duplicate side-effects (jaise double payment) na ho.

### I16 Horizontal vs vertical partitioning

partitioning

**Horizontal Partitioning** (sharding) me table ki rows ko alag-alag databases/servers me split kiya jata hai (jaise user\_id 1-1000 ek shard me, 1001-2000 doosre me) — same schema, different data subsets. **Vertical Partitioning** me table ke columns ko split kiya jata hai (jaise “profile” columns ek jagah, “activity” columns doosri jagah) — different schema pieces, often access-pattern ke basis pe.

## 117 Session management in stateless/distributed systems

sessions

Traditional apps me session data server ki memory me store hoti thi, but distributed systems me (multiple servers, load balancer ke peeche) ye problem create karta hai kyunki agla request kisi aur server pe jaa sakta hai. Solutions: **Sticky Sessions** (LB hamesha same server pe bhejta hai — simple but scaling limits), ya session data ko ek **shared external store** (jaise Redis) me rakho jise koi bhi server access kar sake, ya **JWT** jaise stateless tokens use karo jisme session info khud client ke paas encrypted form me rehti hai.

## Distributed Systems & Real Designs

For senior / SDE-2+ rounds — consensus, distributed transactions, resilience patterns, and full walkthroughs like designing a URL shortener.

### A1 Strong consistency vs eventual consistency

consistency models

**Strong Consistency** ka matlab hai write ke turant baad, koi bhi node se read karo, hamesha latest value milegi — predictable but latency/availability thodi compromise hoti hai (especially multi-region systems me). **Eventual Consistency** ka matlab hai write ke turant baad different nodes different (stale) values dikha sakte hain, but time ke saath sab sync ho jaate hain — high availability aur low latency milti hai, but application ko stale reads handle karne ke liye design karna padta hai.

### A2 Synchronous vs Asynchronous communication

sync    async

**Synchronous** communication me caller request bhejta hai aur response ka wait karta hai (blocking) — jaise ek REST API call — simple to reason about but downstream slow hua toh caller bhi block ho jata hai. **Asynchronous** communication me caller request bhejke aage badh jata hai, response baad me (callback/event/polling se) milta hai — better resource utilization aur resilience, but system design/debugging complex ho jata hai.

### A3 Distributed caching (Redis/Memcached) in architecture

distributed cache    Redis

Distributed cache ek separate, shared caching layer hai (Redis/Memcached cluster) jise multiple application servers access kar sakte hain — isse har server ko apna inconsistent local cache maintain nahi karna padta. Ye database load kaafi kam karta hai aur response time improve karta hai, especially read-heavy workloads me. Redis extra features bhi deta hai jaise persistence, pub-sub, aur rich data structures (lists, sets, sorted sets).

### A4 What is Service Discovery?

service discovery

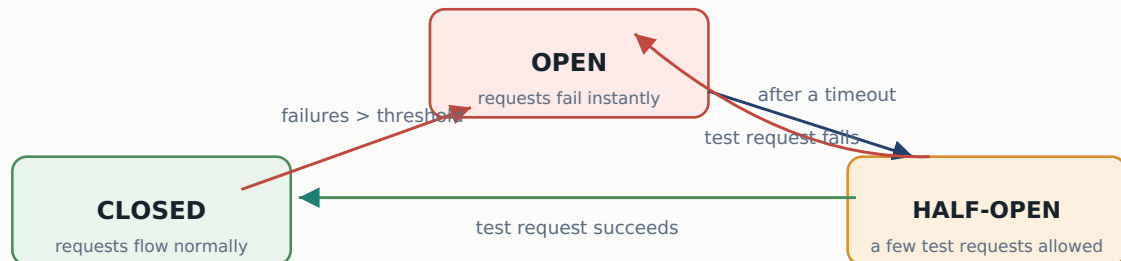
Microservices architecture me services dynamically scale/restart hote rehte hain, unke IP addresses change hote rehte hain — Service Discovery (Consul, Eureka, ya Kubernetes' built-in DNS) ek registry maintain karta hai jisme har service apna location register karta hai, aur doosri services us registry se query karke pata laga sakti hain “ye service abhi kaha hai” — bina hardcoded IPs ke.

## A5 Circuit breaker pattern

circuit breaker resilience

Circuit Breaker cascading failures se bachata hai. Agar koi downstream service baar-baar fail ho raha hai, circuit breaker **OPEN** state me chala jata hai — aage ke requests turant fail ho jate hain (bina actual call kiye), taaki failing service ko recover hone ka time mile. Kuch time baad **HALF-OPEN** state me test requests bheje jate hain — successful hue toh circuit **CLOSED** ho jata hai (normal operation resume).

### DIAGRAM



*Circuit breaker stops a struggling service from being hammered while it recovers, then carefully tests before trusting it again.*

## A6 Leader election in distributed systems

leader election

Distributed systems me kai baar ek node ko “leader” banana padta hai jo coordination decisions leta hai (jaise kaunsa node write accept karega). Leader Election algorithms (Raft, Bully algorithm) ensure karte hain ki sab nodes ek hi leader pe agree karein, aur current leader fail ho jaye toh naya leader automatically choose ho jaye — bina manual intervention ke.

## A7 Consensus algorithms (Paxos/Raft) — high level

Paxos Raft consensus

Consensus algorithms (Paxos, Raft) distributed nodes ko ek single value/decision pe agree karne me help karte hain, chahe kuch nodes fail ho jayein ya messages delay ho jayein. Raft (Paxos se samajhna aasan) me ek leader elect hota hai jo saari writes ko ek consistent “log” me order karta hai aur majority nodes ko replicate karta hai — jab tak majority (quorum) confirm nahi karte, write “committed” nahi mana jata.

## A8 Quorum-based replication (read/write quorum)

quorum

Quorum-based systems me har write ko success maana jata hai jab  $N$  nodes me se kam se kam  $W$  nodes (write quorum) confirm kar dein, aur har read ko  $R$  nodes se collect kiya jata hai (read quorum). Agar  $W + R > N$  ho, toh guarantee milti hai ki read hamesha latest write dekhenga (kyunki overlap hoga). Ye consistency aur availability ke beech tune karne ki flexibility deta hai (Cassandra, DynamoDB isi ka use karte hain).

## A9 Event-driven architecture & Pub-Sub

event-driven pub-sub

Event-Driven Architecture me services events publish karte hain (jaise “OrderPlaced”) jab kuch important hota hai, aur interested services un events ko subscribe karke react karte hain — services directly coupled nahi hote, sirf events ke through communicate karte hain. **Pub-Sub** is pattern ka core mechanism hai — ek publisher ek topic pe bhejta hai, aur us topic ke saare subscribers wo message paate hain (broadcast, unlike point-to-point queue jaha ek message sirf ek consumer process karta hai).

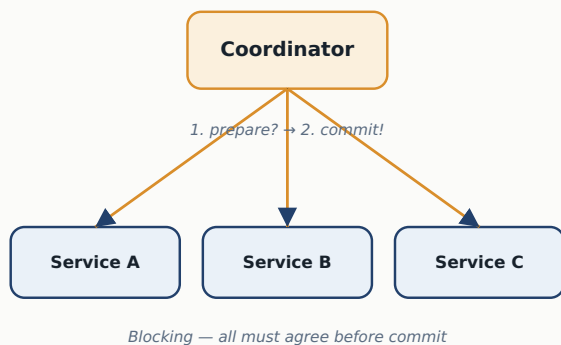
## A10 Distributed transactions — 2PC vs Saga pattern

2PC Saga pattern

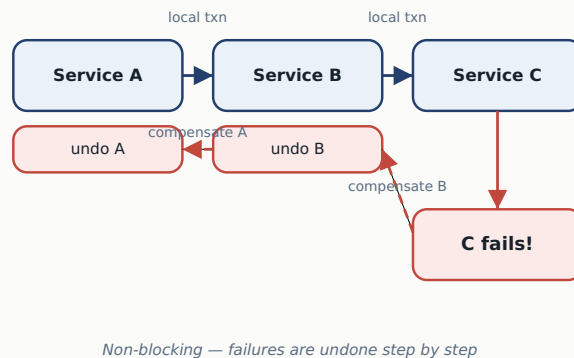
**Two-Phase Commit (2PC):** ek coordinator saare participating services se poochta hai “commit karne ko ready ho?” (prepare phase), sab “yes” bolein tabhi coordinator “commit karo” bolta hai (commit phase) — strong consistency but coordinator single point of failure hai aur services ko lock hold karna padta hai (blocking, slow). **Saga Pattern:** transaction ko chhote local transactions me todha jata hai, har ek apna “compensating action” (undo) define karta hai — beech me koi step fail ho toh pehle wale steps ke compensating actions chalake rollback simulate kiya jata hai — non-blocking aur scalable, but eventual consistency accept karni padti hai.

### DIAGRAM

#### TWO-PHASE COMMIT (2PC)



#### SAGA PATTERN



Non-blocking — failures are undone step by step

## A11 Bloom filters — what and where used?

Bloom filter

Bloom Filter ek probabilistic data structure hai jo bahut memory-efficient tarike se batata hai “ye element set me hai ya nahi” — but ek catch hai: false positives ho sakte hain (“hai” bol sakta hai jabki nahi hai), false negatives kabhi nahi hote. Use cases: database query se pehle check karna ki row exist karti hai ya nahi (disk read bachane ke liye), ya web crawlers me duplicate URLs detect karna.

## A12 Zookeeper — what is it used for?

Zookeeper

Apache Zookeeper ek distributed coordination service hai jo distributed systems ke common problems solve karta hai: configuration management (saare nodes ek jagah se config padhein), leader election, distributed locks, aur service discovery. Kafka jaise systems internally Zookeeper (ya newer KRaft mode) use karte the cluster metadata aur coordination manage karne ke liye.

## A13 Layer 4 vs Layer 7 load balancing

L4 LB L7 LB

**Layer 4** (Transport layer) load balancer sirf IP address aur port dekh kar routing decision leta hai — fast hai kyunki packet content padhna nahi padta, but smart routing (URL path ke basis pe) nahi kar sakta. **Layer 7** (Application layer) load balancer poora HTTP request padh sakta hai (headers, URL path, cookies) — smarter routing decisions le sakta hai (jaise “/api/\*” ek service ko, “/images/\*” doosre ko), but thoda zyada processing overhead hota hai.

## A14 Back-of-the-envelope estimation approach

capacity estimation

System design interviews me estimation ek important skill hai — daily active users, requests per second, storage needs, bandwidth jaise numbers ka rough calculation karna. Approach: pehle assumptions likho (jaise “10 million DAU, har user din me 5 requests karta hai”), phir simple math se QPS, storage (per-record size × total records), aur bandwidth nikaalo. Exact number matter nahi karta — interviewer dekhta hai ki tumhara approach structured hai aur tum sahi order-of-magnitude tak pahunch pa rahe ho.

## A15 Blue-green vs canary deployment

deployment strategies

**Blue-Green Deployment** me do identical production environments hote hain (“blue” = current live, “green” = naya version) — green fully tested ho jaye toh traffic ek saath switch kar diya jata hai, aur kuch galat ho toh turant wapas blue pe switch (instant rollback).

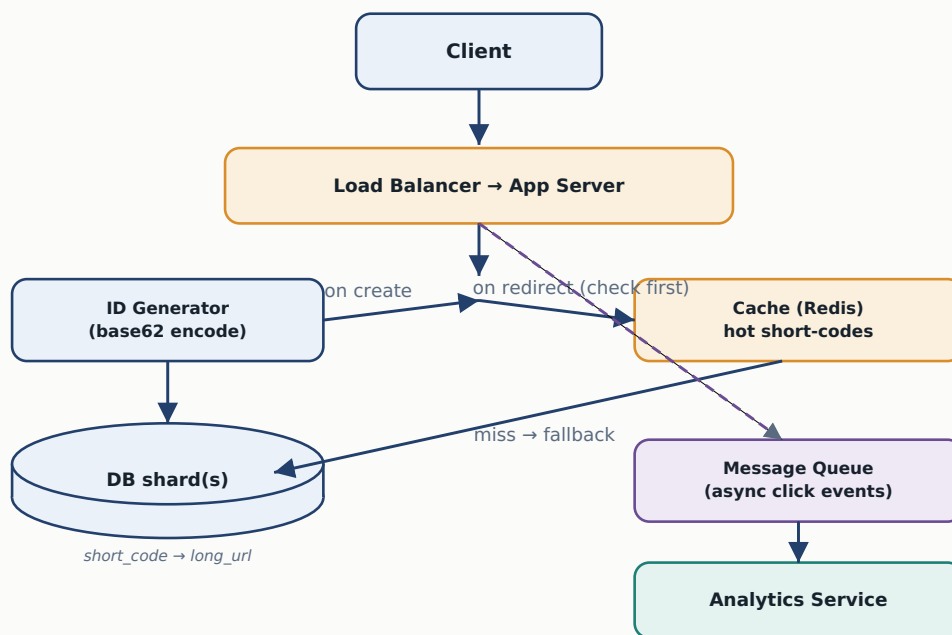
**Canary Deployment** me naya version pehle sirf chhote percentage traffic (jaise 5%) ko dikhaya jata hai, monitor kiya jata hai, aur gradually percentage badhaya jata hai — risk kam hota hai kyunki issues jaldi aur chhote scale pe pakde jate hain.

## A16 Designing a URL shortener (high-level walkthrough)

URL shortener system design walkthrough

Requirements: long URL do, short URL milna chahiye (jaise bit.ly/abc123), aur short URL pe click karne par original URL pe redirect hona chahiye. High-level design: ek unique short code generate karo (base62 encoding of an auto-incrementing ID, ya hash-based approach), database me long-URL ke saath map karke store karo (short code pe index laga ke O(1) lookup ke liye), aur ek caching layer (Redis) add karo popular URLs ke liye taaki database pe load kam ho. Scale ke liye: database shard karo, read replicas add karo (reads >> writes is case me), aur analytics/click-tracking ke liye async event bhejo (message queue) taaki redirect fast rahe.

DIAGRAM



*Reads (redirects) >> Writes (new short URLs) — cache + read replicas do the heavy lifting.*

## Quick Revision Cheat Sheet

Skim this page 10 minutes before you walk into the interview.

|                                |                                                                             |
|--------------------------------|-----------------------------------------------------------------------------|
| <b>Vertical vs Horizontal</b>  | Bigger machine vs more machines — horizontal wins at scale                  |
| <b>CAP theorem</b>             | Pick 2 of Consistency, Availability, Partition Tolerance (really: CP vs AP) |
| <b>ACID vs BASE</b>            | ACID = strict correctness   BASE = available now, consistent eventually     |
| <b>Cache-aside</b>             | App checks cache → miss → read DB → populate cache → return                 |
| <b>Cache eviction</b>          | LRU (recency)   LFU (frequency)   FIFO (insertion order)                    |
| <b>Sharding key</b>            | Choose to spread data evenly & minimize cross-shard queries                 |
| <b>Token vs Leaky bucket</b>   | Token = allows bursts   Leaky = smooths to a constant rate                  |
| <b>Consistent hashing</b>      | Adding/removing a node reshuffles only its neighbouring keys                |
| <b>2PC vs Saga</b>             | 2PC = blocking, strong consistency   Saga = async, compensating actions     |
| <b>Circuit breaker states</b>  | CLOSED → OPEN (fail fast) → HALF-OPEN (test) → CLOSED                       |
| <b>L4 vs L7 load balancing</b> | L4 = IP/port, fast   L7 = HTTP-aware, smarter routing                       |
| <b>Blue-green vs Canary</b>    | Blue-green = instant switch   Canary = gradual rollout                      |

*All the best for your interview — jaake tod do! ★*